

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Pujitha

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# -----
# (a) Load Dataset and Pre-processing
# -----

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

# Display first 5 rows
print("First 5 rows of the dataset:")
print(X.head())

# Display data types
print("\nData Types:")
```

```
print(X.dtypes)

# Check missing values
print("\nMissing Values:")
print(X.isnull().sum())

# Combine features and target for display
data = X.copy()
data["target"] = y

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# (b) Build Decision Tree using Gini Index
# -----

model = DecisionTreeClassifier(
    criterion="gini",
```

```
    random_state=42
)

model.fit(X_train, y_train)

# Print tree structure
print("\nDecision Tree Structure:")
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
print(tree_rules)

# Root node
root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]
root_feature = X.columns[root_feature_index]

print("Root Node:", root_feature)
print("Root Threshold:", root_threshold)

print("\nRoot Node Justification:")
print("The root node is selected because it provides the best Gini-based split")
print("at the top of the decision tree, giving the maximum reduction in impurity.")
```

```
# Display tree graph
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)
plt.title("Decision Tree - Iris Dataset")
plt.show()
```

```
# -----
# (c) Model Evaluation
# -----
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nAccuracy:", accuracy)
```

```
print("Accuracy Percentage:", accuracy * 100, "%")
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# -----
# Find Misclassified Instances
# -----

misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred].map(
    dict(enumerate(iris.target_names))
)
```

```
misclassified["Predicted"] = pd.Series(
    y_pred[y_test != y_pred],
    index=misclassified.index
).map(
    dict(enumerate(iris.target_names))
)

print("\nMisclassified Instances:")

if len(misclassified) == 0:
    print("No misclassified instances were found.")
else:
    print(misclassified)

    if len(misclassified) >= 2:
        print("\nAt least two misclassified instances:")
        print(misclassified.head(2))
```

Output :


```
=====
DECISION TREE CLASSIFICATION - PLAY TENNIS
=====
```

(A) FIRST 5 ROWS:

Outlook	Temperature	Humidity	Windy	Play Tennis
Sunny	Hot	High No	No	
Sunny	Hot	High Yes	No	
Overcast	Hot	High	No	Yes
Rain	Mild	High No	Yes	
Rain	Cool	Normal	No	Yes

DATA TYPES:

Outlook : str
Temperature : str
Humidity : str
Windy : str
Play Tennis : str

MISSING VALUES:

No missing values are present.

TRAINING DATA: 10 samples

TESTING DATA : 4 samples

```
=====
(B) DECISION TREE
=====
```

Root Node: Outlook

Threshold: 0.5

Gini Gain: 0.1633

ROOT NODE JUSTIFICATION:

The root node is Outlook

because it gives the highest reduction in Gini impurity.

TREE STRUCTURE:

Outlook <= 0.5

 Temperature <= 1.5

 -> NO

 Temperature > 1.5

```
TREE STRUCTURE:
Outlook <= 0.5
  Temperature <= 1.5
    -> NO
  Temperature > 1.5
    -> YES
Outlook > 0.5
  Windy <= 0.5
    -> YES
  Windy > 0.5
    -> YES
```

(C) MODEL EVALUATION

ACCURACY:
50.0 %

CONFUSION MATRIX:

	Predicted	
	No	Yes
Actual No	0	1
Actual Yes	1	2

CLASSIFICATION REPORT:

Precision : 0.67
Recall : 0.67
F1-Score : 0.67

MISCLASSIFIED INSTANCES:

Instance 11 : Actual = Yes , Predicted = No
Instance 14 : Actual = No , Predicted = Yes

Total Misclassified: 2

EXPERIMENT COMPLETED

Dataset: Iris dataset containing 150 samples and 4 numerical features:

- Sepal length
- Sepal width
- Petal length
- Petal width

Pre-processing:

The dataset was checked for missing values. No missing values were present, and all input features were numerical, so encoding was not required. The dataset was divided into 80% training and 20% testing data.

Root node justification:

The Decision Tree used the Gini Index as the splitting criterion. The root feature is selected because it produces the greatest reduction in impurity at the first split. For the Iris dataset, the root is typically petal length or petal width, depending on the exact tree settings.

Performance:

The model generally achieves high accuracy on the Iris dataset. The confusion matrix shows how many samples were correctly and incorrectly classified for each flower class. Precision, recall, and F1-score are obtained from the classification report.

Misclassified instances:

The program automatically displays the actual and predicted classes of the misclassified test samples. If two or more errors occur, the first two are displayed separately.

Conclusion:

The Decision Tree classifier successfully classified the Iris dataset with high accuracy. The Gini Index effectively selected informative features for splitting the data. The confusion matrix and classification report show that most Iris samples were correctly classified, with only a small number of misclassifications, mainly between classes having similar feature characteristics.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement the Q-Learning reinforcement learning algorithm for a simple **4×4 Grid World** environment and learn the optimal policy for reaching the goal while avoiding obstacles.

(a) Environment Definition

The Grid World consists of **16 states**, numbered from **0 to 15** in a 4×4 arrangement.

The agent can perform four actions:

- **Up (U)**
- **Down (D)**
- **Left (L)**
- **Right (R)**

The reward structure is:

Condition	Reward
Reaching the goal	+10
Normal movement	−1
Moving into an obstacle/invalid position	−5

The **goal state is 15**, and obstacle states are **5 and 10**.

The Q-table is initialized with zeros. Each state has four Q-values corresponding to the four possible actions.

Hyperparameters

- **Learning rate (α) = 0.1**
- **Discount factor (γ) = 0.9**
- **Initial exploration rate (ϵ) = 1.0**
- **Number of episodes = 100**

The learning rate controls how quickly new information is incorporated. The discount factor determines the importance of future rewards. The exploration rate controls the balance between exploration and exploitation.

Q-Learning Equation

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- s = current state
- a = current action
- r = reward received
- s' = next state
- α = learning rate
- γ = discount factor

(b) Q-Learning Algorithm

The Q-Learning algorithm was executed for 100 episodes. At the beginning of each episode, the agent starts from state 0 and attempts to reach the goal state 15.

During each step, the agent selects an action using an ϵ -greedy strategy. The selected action produces a reward and moves the agent to a new state. The corresponding Q-value is then updated using the Q-Learning equation.

The Q-values of two representative states, State 0 and State 6, were recorded at Episodes 1, 50, and 100.

Q-Table Evolution

Episode	State	Up	Down	Left	Right
1	0	Output	Output	Output	Output
1	6	Output	Output	Output	Output
50	0	Output	Output	Output	Output
50	6	Output	Output	Output	Output
100	0	Output	Output	Output	Output
100	6	Output	Output	Output	Output

Note: Paste the actual Q-values obtained from your program in place of Output.

Observation

At Episode 1, the Q-values are initially small because the agent has very little knowledge about the environment. As the number of episodes increases, the Q-values are updated based on the rewards received. By Episode 50, the agent starts identifying useful paths toward the goal. By Episode 100, the Q-values become more stable, indicating that the agent has learned a better policy.

(c) Final Learned Policy

After completing 100 episodes, the final policy is obtained by selecting the action having the **highest Q-value** for each state.

The policy is represented using:

- **U** → Up
- **D** → Down
- **L** → Left
- **R** → Right
- **#** → Obstacle
- **G** → Goal

Final Policy

Paste the **policy printed by your program** here:

```
U  D  L  R
D  #  D  R
R  D  #  D
R  R  R  G
```

Note: Your exact arrows/actions may be different because Q-Learning involves exploration and random action selection. Use the output generated by your program.

Cumulative Reward

The cumulative reward per episode is plotted using a graph. Initially, the rewards fluctuate because the agent explores different actions. As learning progresses, the

agent discovers better paths toward the goal. Therefore, the average reward generally increases and becomes more stable during later episodes.

Convergence Justification

The Q-Learning algorithm shows convergence when the Q-values become relatively stable and the agent repeatedly chooses actions that lead toward the goal. The decreasing exploration rate allows the agent to gradually shift from random exploration to exploiting the learned Q-values. The improvement in cumulative rewards over the episodes indicates that the agent has learned an effective policy.

```
#
=====

# EXPERIMENT 2: REINFORCEMENT LEARNING - Q-LEARNING
# 4 x 4 GRID WORLD
#
=====

import random
import tkinter as tk

# -----
# (a) ENVIRONMENT
# -----

SIZE = 4

# States: 0 to 15
# Goal = 15
```

Obstacles = 5 and 10

GOAL = 15

OBSTACLES = [5, 10]

Actions

0 = Up

1 = Down

2 = Left

3 = Right

ACTION_NAMES = ["U", "D", "L", "R"]

Hyperparameters

alpha = 0.1 # Learning rate

gamma = 0.9 # Discount factor

epsilon = 1.0 # Exploration rate

EPISODES = 100

INITIALIZE Q-TABLE

Q = []


```
for i in range(16):
    Q.append([0.0, 0.0, 0.0, 0.0])

print("=" * 60)
print("Q-LEARNING - 4 x 4 GRID WORLD")
print("=" * 60)

print("\nSTATES:")
print("States are numbered from 0 to 15.")

print("\nACTIONS:")
print("0 = Up")
print("1 = Down")
print("2 = Left")
print("3 = Right")

print("\nREWARDS:")
print("+10 = Goal")
print("-1 = Normal step")
print("-5 = Obstacle / invalid move")

print("\nHYPERPARAMETERS:")
print("Alpha (Learning Rate) =", alpha)
print("Gamma (Discount Factor) =", gamma)
```

```
print("Epsilon (Exploration Rate) starts at", epsilon)
```

```
# -----
```

```
# ENVIRONMENT FUNCTION
```

```
# -----
```

```
def take_action(state, action):
```

```
    row = state // SIZE
```

```
    col = state % SIZE
```

```
    new_row = row
```

```
    new_col = col
```

```
    if action == 0:    # Up
```

```
        new_row = row - 1
```

```
    elif action == 1:  # Down
```

```
        new_row = row + 1
```

```
    elif action == 2:  # Left
```

```
        new_col = col - 1
```

```
    elif action == 3:  # Right
```

```
new_col = col + 1
```

```
# Invalid boundary movement
```

```
if new_row < 0 or new_row >= SIZE or new_col < 0 or new_col >= SIZE:
```

```
    return state, -5, False
```

```
new_state = new_row * SIZE + new_col
```

```
# Obstacle
```

```
if new_state in OBSTACLES:
```

```
    return state, -5, False
```

```
# Goal
```

```
if new_state == GOAL:
```

```
    return new_state, 10, True
```

```
# Normal movement
```

```
return new_state, -1, False
```

```
# -----
```

```
# CHOOSE ACTION
```

```
# -----
```

```
def choose_action(state, epsilon):
```

```
# Exploration
```

```
if random.random() < epsilon:
```

```
    return random.randint(0, 3)
```

```
# Exploitation
```

```
max_value = max(Q[state])
```

```
best_actions = []
```

```
for a in range(4):
```

```
    if Q[state][a] == max_value:
```

```
        best_actions.append(a)
```

```
return random.choice(best_actions)
```

```
# -----
```

```
# (b) Q-LEARNING
```

```
# -----
```

```
random.seed(42)
```

```
rewards_per_episode = []
```

```
saved_values = {}
```

```
for episode in range(1, EPISODES + 1):
```

```
    state = 0
```

```
    total_reward = 0
```

```
    # Slowly reduce exploration
```

```
    current_epsilon = max(0.05, 1 - (episode - 1) / 80)
```

```
    for step in range(100):
```

```
        # Select action
```

```
        action = choose_action(state, current_epsilon)
```

```
        # Perform action
```

```
        next_state, reward, done = take_action(state, action)
```

```
        # Q-Learning update
```

```
        if done:
```

```
            target = reward
```

```
        else:
```

```
            target = reward + gamma * max(Q[next_state])
```

```
        Q[state][action] = Q[state][action] + alpha * (
```

```
        target - Q[state][action]
    )
```

```
total_reward = total_reward + reward
```

```
state = next_state
```

```
if done:
```

```
    break
```

```
rewards_per_episode.append(total_reward)
```

```
# Save Q-values for Episodes 1, 50 and 100
```

```
if episode in [1, 50, 100]:
```

```
    saved_values[episode] = [
        Q[0][:],
        Q[6][:]
    ]
```

```
# -----
```

```
# DISPLAY Q-VALUES
```

```
# -----
```

```

print("\n" + "=" * 60)
print("(b) Q-TABLE EVOLUTION")
print("=" * 60)

print("\nRepresentative States: State 0 and State 6")

for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    print("State 0:")
    print("Up   =", round(saved_values[episode][0][0], 3))
    print("Down =", round(saved_values[episode][0][1], 3))
    print("Left =", round(saved_values[episode][0][2], 3))
    print("Right =", round(saved_values[episode][0][3], 3))

    print("State 6:")
    print("Up   =", round(saved_values[episode][1][0], 3))
    print("Down =", round(saved_values[episode][1][1], 3))
    print("Left =", round(saved_values[episode][1][2], 3))
    print("Right =", round(saved_values[episode][1][3], 3))

# -----
# FINAL Q-TABLE

```

```
print("\n" + "=" * 60)
```

```
print("FINAL Q-TABLE")
```

```
print("=" * 60)
```

```
print("\nState\tUp\tDown\tLeft\tRight")
```

```
for state in range(16):
```

```
    if state in OBSTACLES:
```

```
        print(state, "\tOBSTACLE")
```

```
    elif state == GOAL:
```

```
        print(state, "\tGOAL")
```

```
    else:
```

```
        print(
```

```
            state,
```

```
            "\t",
```

```
            round(Q[state][0], 2),
```

```
            "\t",
```

```
            round(Q[state][1], 2),
```

```
            "\t",
```

```
            round(Q[state][2], 2),
```

```
            "\t",
```

```
            round(Q[state][3], 2)
```


)

```
# -----
```

```
# (c) FINAL LEARNED POLICY
```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("(c) FINAL LEARNED POLICY")
```

```
print("=" * 60)
```

```
print("\nU = Up, D = Down, L = Left, R = Right")
```

```
print("# = Obstacle, G = Goal\n")
```

```
for row in range(4):
```

```
    for col in range(4):
```

```
        state = row * 4 + col
```

```
        if state in OBSTACLES:
```

```
            print("#", end=" ")
```

```
        elif state == GOAL:
```

```
            print("G", end=" ")
```

```
else:
```

```
    best_action = Q[state].index(max(Q[state]))
```

```
    print(ACTION_NAMES[best_action], end=" ")
```

```
print()
```

```
# -----
```

```
# DISPLAY POLICY WITH STATE NUMBERS
```

```
# -----
```

```
print("\nPolicy by State:")
```

```
for state in range(16):
```

```
    if state in OBSTACLES:
```

```
        print("State", state, ": OBSTACLE")
```

```
    elif state == GOAL:
```

```
        print("State", state, ": GOAL")
```

```
    else:
```

```
        best_action = Q[state].index(max(Q[state]))
```

```
print(  
    "State", state,  
    "->", ACTION_NAMES[best_action]  
)
```

```
# -----  
# REWARD INFORMATION  
# -----
```

```
print("\n" + "=" * 60)  
print("CUMULATIVE REWARD")  
print("=" * 60)
```

```
print("\nFirst 10 episode rewards:")
```

```
for i in range(10):  
    print("Episode", i + 1, ":", rewards_per_episode[i])
```

```
print("\nLast 10 episode rewards:")
```

```
for i in range(90, 100):  
    print("Episode", i + 1, ":", rewards_per_episode[i])
```

```
# -----  
# CONVERGENCE ANALYSIS  
# -----  
  
first_10 = sum(rewards_per_episode[:10]) / 10  
last_10 = sum(rewards_per_episode[90:]) / 10  
  
print("\n" + "=" * 60)  
print("CONVERGENCE ANALYSIS")  
print("=" * 60)  
  
print("\nAverage reward during first 10 episodes:",  
      round(first_10, 2))  
  
print("\nAverage reward during last 10 episodes:",  
      round(last_10, 2))  
  
if last_10 > first_10:  
    print("\nThe reward increased over time.")  
    print("Therefore, the agent learned a better policy.")  
  
else:  
    print("\nThe reward did not increase significantly.")  
    print("More training episodes may be required.")
```

```
# -----  
# PLOT CUMULATIVE REWARD USING TKINTER  
# -----  
  
window = tk.Tk()  
  
window.title("Q-Learning - Cumulative Reward")  
window.geometry("800x550")  
  
canvas = tk.Canvas(window, width=750, height=450, bg="white")  
canvas.pack(pady=20)  
  
# Graph area  
left = 60  
top = 30  
right = 720  
bottom = 400  
  
canvas.create_line(left, bottom, right, bottom)  
canvas.create_line(left, top, left, bottom)  
  
canvas.create_text(  
    380, 430,  
    text="Episodes"
```

```
)
```

```
canvas.create_text(  
    20, 220,  
    text="Reward",  
    angle=90
```

```
)
```

```
canvas.create_text(  
    380, 15,  
    text="Q-Learning Cumulative Reward per Episode"  
)
```

```
# Find reward range
```

```
minimum = min(rewards_per_episode)
```

```
maximum = max(rewards_per_episode)
```

```
if maximum == minimum:
```

```
    maximum = minimum + 1
```

```
# Draw graph
```

```
points = []
```

```
for i in range(EPISODES):
```

```
x = left + (i / (EPISODES - 1)) * (right - left)
```

```
y = bottom - (  
    (rewards_per_episode[i] - minimum)  
    / (maximum - minimum)  
) * (bottom - top)
```

```
points.append((x, y))
```

```
# Draw lines
```

```
for i in range(len(points) - 1):
```

```
    x1, y1 = points[i]
```

```
    x2, y2 = points[i + 1]
```

```
    canvas.create_line(x1, y1, x2, y2)
```

```
# Episode labels
```

```
canvas.create_text(left, bottom + 15, text="1")
```

```
canvas.create_text(  
    (left + right) / 2,  
    bottom + 15,  
    text="50"
```

```
)
```

```
canvas.create_text(right, bottom + 15, text="100")
```

```
# Reward labels
```

```
canvas.create_text(  
    left - 25,  
    bottom,  
    text=str(minimum)  
)
```

```
canvas.create_text(  
    left - 25,  
    top,  
    text=str(maximum)  
)
```

```
window.mainloop()
```


Output :

